

Contents

MANUAL COMPLETO — Fábrica Agêntica de Publicações + Máquina de Vendas	1
ÍNDICE	1
1. VISÃO GERAL	1
2. TIPOS DE OBRA (8 tipos)	2
3. MÁQUINA DE VENDAS (projeto deployável)	4
4. COMANDOS DISPONÍVEIS (13 comandos)	9
5. SCRIPTS DETERMINÍSTICOS (43 scripts)	10
6. SKILLS (26 skills)	12
7. SUBAGENTES (7 agentes)	13
8. TEMPLATES (12 templates)	13
9. SPECS (9 specs)	14
10. FLUXO COMPLETO DE UMA OBRA	14
11. FLUXO DA MÁQUINA DE VENDAS	15
12. CONFIGURAÇÃO E DEPLOY	16
13. ROTEAMENTO DE LLMs	17
14. TROUBLESHOOTING	18
15. ESTRUTURA DE OUTPUT	18

MANUAL COMPLETO — Fábrica Agêntica de Publicações + Máquina de Vendas

Tudo que é possível criar, como criar, e o que cada ferramenta faz.

ÍNDICE

1. Visão Geral
 2. Tipos de Obra (8 tipos)
 3. Máquina de Vendas (projeto deployável)
 4. Comandos Disponíveis (13 comandos)
 5. Scripts Determinísticos (43 scripts)
 6. Skills (26 skills)
 7. Subagentes (7 agentes)
 8. Templates (12 templates)
 9. SPECS (9 specs)
 10. Fluxo Completo de uma Obra
 11. Fluxo Completo da Máquina de Vendas
 12. Configuração e Deploy
 13. Roteamento de LLMs
 14. Troubleshooting
-

1. VISÃO GERAL

A Fábrica Agêntica de Publicações é um sistema autônomo que:

1. **Cria conteúdo** (livros, TCCs, artigos, e-books, playbooks, lead magnets, decks, e-mails)
2. **Gera artefatos** (PDF, EPUB, PPTX, áudio, vídeos, artes)
3. **Monta máquinas de vendas** (projeto full-stack deployável com frontend, backend, banco, automação)
4. **Opera 24/7** (busca leads, envia e-mails, monitora métricas, auto-corrige)

O que pode ser criado:

Categoria	Itens	Quantidade
Conteúdo	Livro, TCC, Artigo, E-book, Playbook, Lead Magnet, Deck, E-mails	8 tipos
Formatos	PDF, EPUB, PPTX, HTML, Markdown, Áudio (MP3), Vídeo (MP4), PNG	8 formatos
Marketing	Páginas de venda, captura, e-mails, posts, stories, DMs	6 artefatos
Deploy	Frontend Next.js, Backend FastAPI, SQLite, Docker, Vercel	5 peças
Automação	Lead hunter, e-mail sender, funnel monitor, auto-correct	4 scripts

2. TIPOS DE OBRA (8 tipos)

2.1 LIVRO (/criar-livro)

O que é: Livro técnico completo, formatado em ABNT, com capítulos paralelos.

O que é gerado: - 16+ capítulos com estrutura EITA-V2 (Introdução, Explica, Ilustra, Técnica, Aplica, Conclusão, Referências) - Diagramas Mermaid por capítulo - Código validado por capítulo - PDF final via Pandoc→Typst - EPUB para e-readers - Capa 2D plano com badge de nível

Como criar:

/criar-livro Observabilidade em Sistemas Distribuídos com OpenTelemetry

Fluxo automático: 1. Pesquisador varre a web → gera dossiê técnico 2. Arquiteto desenha sumário macro (Partes → Capítulos) 3. Estrategista decompõe cada capítulo em 3 pilares 4. Redatores escrevem capítulos em paralelo (lotes de 4) 5. Revisor técnico audita (terminologia, truncamento, código) 6. Compilador ABNT merge + gera PDF

Spec: SPEC.md — 14 requisitos obrigatórios (R1-R14)

Tempo estimado: 30-60 minutos (autônomo)

2.2 TCC (/criar-tcc)

O que é: Trabalho de Conclusão de Curso formatado em ABNT (NBR 14724).

O que é gerado: - Seções acadêmicas (Introdução, Referencial Teórico, Metodologia, Resultados, Conclusão) - Citações autor-data (NBR 10520) - Folha de aprovação - Resumo/Abstract (NBR 6028) - Sumário (NBR 6027) - PDF formatado via Pandoc→Typst com template_tcc.typ

Como criar:

/criar-tcc Machine Learning aplicado a detecção de fraudes financeiras

Spec: SPEC_TCC.md

2.3 ARTIGO (/criar-artigo)

O que é: Artigo científico no formato IMRaD (Introdução, Método, Resultados, Discussão).

O que é gerado: - 4 seções IMRaD - Resumo/Abstract (NBR 6028) - Referências (NBR 6023) - PDF via Pandoc→Typst com `template_artigo.typ`

Como criar:

`/criar-artigo` Redes Neurais Convolucionais para classificação de imagens médicas

Spec: `SPEC_ARTIGO.md`

Custo LLM: Baixo (compressão, não geração)

2.4 E-BOOK (`/criar-ebook`)

O que é: Versão comercial-leve de um livro existente.

O que é gerado: - Adaptação de tom (parágrafos curtos, mais subtítulos, sem citação numerada) - CTA final - PDF estilizado

Como criar:

`/criar-ebook` observabilidade-sistemas-distribuidos

Requer: Livro já criado (deriva do livro-mãe)

Spec: `SPEC_EBOOK.md`

2.5 PLAYBOOK (`/criar-playbook`)

O que é: Guia prático com passos de bancada, extraído automaticamente.

O que é gerado: - Passos práticos extraídos do conteúdo - Cards de ação - Checklist - PDF com template de cards (`template_playbook.typ`)

Como criar:

`/criar-playbook` observabilidade-sistemas-distribuidos

Custo LLM: Zero (extração determinística via `extrair-passos-praticos.py`)

Spec: `SPEC_PLAYBOOK.md`

2.6 LEAD MAGNET (`/criar-lead-magnet`)

O que é: Material gratuito para captura de leads.

O que é gerado: - Conteúdo resumido e atraente - PDF via HTML+CSS→Chromium (`gerar-lead-magnet-pdf.py`) - Template A4 com CTA no rodapé (`template_lead_magnet.html`)

Como criar:

`/criar-lead-magnet` observabilidade-sistemas-distribuidos

Custo LLM: Zero (extração determinística)

Spec: `SPEC_LEAD_MAGNET.md`

2.7 DECK (`/criar-deck`)

O que é: Apresentação de slides.

O que é gerado: - Slides em formato 16:9 - PDF via Pandoc→Typst com `template_deck.typ` - PPTX editável via Pandoc writer nativo (`gerar-pptx.py`)

Como criar:

/criar-deck observabilidade-sistemas-distribuidos

Spec: SPEC_DECK.md

2.8 E-MAILS (/criar-emails)

O que é: Sequência de e-mails de divulgação/marketing.

O que é gerado: - Sequência de 5-7 e-mails - Templates HTML para cada e-mail - Cronograma de envio

Como criar:

/criar-emails observabilidade-sistemas-distribuidos

Spec: SPEC_EMAILS.md

2.9 COLEÇÃO (/colecacao)

O que é: Conjunto de todos os formatos derivados de uma mesma obra.

O que é gerado: - Manifesto JSON em output/_colecacoes/<nome>.json - Sincronização de todos os artefatos (livro + ebook + playbook + deck + lead magnet + e-mails) - Identidade visual compartilhada

Como criar:

/colecacao observabilidade-sistemas-distribuidos

2.10 PRODUÇÃO COMPLETA (/produzir-obra-completa)

O que é: Executa todos os tipos acima em sequência para uma mesma obra.

O que é gerado: - Livro → E-book → Playbook → Lead Magnet → Deck → E-mails → Coleção

Como criar:

/produzir-obra-completa Inteligência Artificial para Empreendedores

2.11 MEGA-LIVRO (/compilar-mega-livro)

O que é: Unifica múltiplos livros em um único mega-livro.

O que é gerado: - Numeração sequencial unificada - Prefácio e conclusão geral - Sumário dinâmico - PDF via Pandoc+Typst

Como criar:

/compilar-mega-livro

3. MÁQUINA DE VENDAS (projeto deployável)

3.1 O QUE É

Cada obra pode gerar um **projeto full-stack completo** com:

Componente	Tecnologia	O que faz
Frontend	Next.js 14	Landing pages, captura, admin dashboard
Backend	FastAPI	APIs de leads, e-mails, métricas, webhooks
Database	SQLite	Leads, interações, vendas, campanhas
Scripts	Python	Lead hunter, e-mail sender, monitor, auto-correct
Deploy	Docker + Vercel	Deploy local ou cloud

3.2 COMO CRIAR

`/criar-maquina observabilidade-sistemas-distribuidos`

Ou via script:

`python scripts/criar-maquina-vendas.py <slug> --tipo completo`

Importante: após gerar, a máquina nasce com **copy genérica de demonstração** (“Autor Digital”, “centenas de pessoas”) — a personalização por nicho é obrigatória antes de publicar (ver seção 3.10).

3.3 TIPOS DE MÁQUINA

Tipo	Componentes	Quando usar
completo	Frontend + Backend + DB + Scripts + Deploy	Padrão
parcial	Frontend + Backend + DB	Sem automação
landing	Apenas Frontend	Landing page pura
backend	Apenas Backend + DB	API pura

3.4 O QUE É GERADO (83 arquivos)

```
marketing/maquinas/{slug}/
├─ frontend/           (29 arquivos) Next.js completo
├─ backend/           (16 arquivos) FastAPI completo
├─ database/          (2 arquivos) SQLite schema + seed
├─ scripts/           (5 arquivos) Automação
├─ config/            (8 arquivos) Configurações
├─ templates/         (8 arquivos) E-mails, posts, DMs
├─ .claude/           (subagentes, skills, commands)
├─ docker-compose.yml  Deploy Docker
├─ vercel.json         Deploy Vercel
├─ AGENTS.md          Orquestrador
├─ CLAUDE.md          Regras do agente
├─ SPEC.md            Spec completa
└─ README.md          Manual de deploy
```

3.5 FRONTEND (Next.js 14)

Rota	Página	Função
/	Página de Venda	Hero, dor, solução, stack de valor, preço, garantia, CTA
/captura	Captura de Lead	Formulário nome + e-mail, sem distrações
/obrigado	Agradecimento	Confirmação + oferta tripwire
/checkout	Checkout	Pagamento (Stripe/Kiwify)
/admin	Dashboard	Métricas gerais
/admin/leads	Leads	Tabela com filtros e busca
/admin/emails	E-mails	Sequências e status
/admin/metricas	Métricas	Funil de conversão
/api/lead	API Lead	POST: cadastro de lead
/api/checkout	API Checkout	POST: registra pedido + lead no backend, devolve link de pagamento (R11)
/api/webhook	API Webhook	POST: pagamento
/api/health	API Health	GET: status

Não remover a rota `/api/checkout` — o `checkout/page.tsx` posta nela; sem ela o checkout quebra com 404. Ela lê `BACKEND_URL/NEXT_PUBLIC_BACKEND_URL` (fallback `http://127.0.0.1:8000`) e registra o lead em `/api/leads/`.

3.6 BACKEND (FastAPI)

Endpoint	Método	Função
/api/leads	POST	Cadastrar lead
/api/leads	GET	Listar leads (filtros: estágio, score, fonte)
/api/leads/{id}	GET	Buscar lead por ID
/api/leads/{id}/qualificar	POST	Scoring automático (0-100)
/api/emails/enviar	POST	Disparar sequência de e-mails
/api/emails/enviados	GET	Listar e-mails enviados
/api/funil/metricas	GET	Métricas do funil
/api/funil/auto-correct	POST	Disparar auto-correção
/api/funil/relatorio	GET	Relatório diário
/api/webhook/pagamento	POST	Webhook Stripe/Kiwify
/health	GET	Health check

3.7 DATABASE (SQLite)

Tabela	Campos	Função
leads	id, nome, email, score, fonte, estágio	Cadastro de leads
interações	id, lead_id, tipo, página, data	Tracking de comportamento
vendas	id, lead_id, produto, valor, status	Registro de vendas
campanhas	id, nome, status, métricas	Gestão de campanhas
emails_enviados	id, lead_id, template, status, data	Log de envios
metricas_diarias	data, visitantes, captura, vendas, receita	Dashboard

3.8 SCRIPTS DE AUTOMAÇÃO

Script	O que faz	Frequência
lead_hunter.py	Busca leads no Instagram por hashtags	3x/dia
email_sender.py	Dispara sequência de e-mails	Diário
funnel_monitor.py	Monitora métricas do funil	Contínuo
auto_correct.py	Corrige páginas com baixa conversão	Quando necessário
deploy.sh	Deploy Docker/Vercel/VPS	Sob demanda

3.9 CONFIGURAÇÕES

Arquivo	O que configura
config/produtos.json	Escada de valor (R\$0 → R\$297)
config/funis.json	Funis de venda (A, B, C)
config/personas.json	ICPs/personas
config/canais.json	Instagram, e-mail, WhatsApp
config/email.json	SMTP (host, porta, credenciais)
config/pagamento.json	Stripe/Kiwify
config/roteamento_modelos.json	Roteamento LLM por tier
config/subagentes.json	Registry de subagentes

3.10 PERSONALIZAÇÃO POR NICHOS (OBRIGATÓRIA)

O template nasce com copy genérica de demonstração. Antes de publicar a máquina, personalizar em **todos** os pontos abaixo pelos termos do nicho da obra de origem:

Área	Arquivos	O que trocar
Configs	config/produtos.json, personas.json, funis.json, canais.json, email.json	Escada de valor, persona, funis, hashtags e remetente do nicho
Frontend	app/page.tsx, components/ Hero.tsx, PricingCard.tsx, app/layout.tsx, app/admin/ layout.tsx, app/captura/ page.tsx	Headline, dor/solução, CTA, metadata
E-mails	templates/emails/*.html	Copy de boas-vindas, nutrição, venda, reativação
Docs	README.md	Apresentação no nicho

Gate de verificação (R12):

```
grep -rn 'Autor Digital\\centenas de pessoas' frontend/app frontend/components
templates/ README.md
# deve retornar VAZIO
```

Teste do checkout (R11):

```
curl -s -X POST http://localhost:3000/api/checkout \
  -H "Content-Type: application/json" \
  -d '{"nome": "Dra. Teste", "email": "teste@exemplo.com", "produto": "obra"}'
# Esperado: {"success":true,...,"valor":97}
# Confirmar o lead no backend:
curl http://localhost:8000/api/leads
```

Alinhar o produto default do checkout: o default(...) da rota deve usar o slug real do produto core em config/produtos.json (senão o funil agrupa por um produto inexistente). Checar com:

```
python -c "import json; d=json.load(open('config/produtos.json',encoding='utf-8')); \
[print(p['slug'], '|', p['preco']) for p in d['produtos']]"
```

Máquinas criadas antes do fix: se a rota /api/checkout não existe ou o checkout/page.tsx usa <form action method="POST"> urlencoded vazio (retorna 500), copiar os arquivos corrigidos do template (templates/maquina/frontend/ app/api/checkout/ e app/checkout/page.tsx), resolver os placeholders e manter a copy do nicho.

4. COMANDOS DISPONÍVEIS (13 comandos)

Comando	O que faz	Exemplo
/criar-livro	Cria livro técnico completo	/criar-livro Observabilidade com OpenTelemetry
/criar-tcc	Cria TCC formatado ABNT	/criar-tcc ML para detecção de fraudes
/criar-artigo	Cria artigo científico IMRaD	/criar-artigo Redes Neurais para imagens médicas
/criar-ebook	Cria e-book a partir de livro	/criar-ebook observabilidade-sistemas
/criar-playbook	Cria playbook prático	/criar-playbook observabilidade-sistemas
/criar-lead-magnet	Cria material de captura	/criar-lead-magnet observabilidade-sistemas
/criar-deck	Cria apresentação de slides	/criar-deck observabilidade-sistemas
/criar-emails	Cria sequência de e-mails	/criar-emails observabilidade-sistemas
/criar-maquina	Cria máquina de vendas deployável	/criar-maquina observabilidade-sistemas
/colecacao	Sincroniza coleção de artefatos	/colecacao observabilidade-sistemas
/produzir-obra-completa	Executa todos os tipos	/produzir-obra-completa IA para Empreendedores
/compilar-mega-livro	Unifica livros em mega-livro	/compilar-mega-livro
/esbocar	Esboço inicial de tema	/esbocar Marketing Digital

5. SCRIPTS DETERMINÍSTICOS (43 scripts)

5.1 Scripts de Geração

Script	O que faz	Uso
gerar-capa.py	Gera capa 2D plano	Automático pós-compilação
gerar-epub.py	Converte MD→EPUB	Automático pós-compilação
gerar-deck.py	Gera deck de slides	/criar-deck
gerar-pptx.py	Exporta PPTX editável	/criar-deck
gerar-lead-magnet.py	Gera lead magnet	/criar-lead-magnet
gerar-lead-magnet-pdf.py	PDF via HTML+Chromium	/criar-lead-magnet
gerar-sequencia-emails.py	Gera sequência de e-mails	/criar-emails
gerar-ilustracoes.py	Gera ilustrações	Automático por capítulo
criar-maquina-vendas.py	Cria máquina de vendas	/criar-maquina
gerar-relatorio-sessao.py	Gera relatório de sessão MD+PDF (V5.2)	Final de sessão (skill gerar-relatorio-sessao)

5.2 Scripts de Validação

Script	O que valida	Quando roda
auditar-obra.py	14 requisitos (R1-R14)	Fase 2.5
validar-codigo.py	Sintaxe de código	Fase 2.5
validar-abnt-tcc.py	Formatação ABNT TCC	Fase 2.5
validar-capa-nivel.py	Badge de nível na capa	Pós-geração
validar-capa-texto.py	Texto da capa	Pós-geração
validar-deck.py	Estrutura do deck	Pós-geração
validar-emails.py	Formato dos e-mails	Pós-geração
validar-lead-magnet.py	Lead magnet	Pós-geração
validar-playbook.py	Playbook	Pós-geração

5.3 Scripts de Processamento

Script	O que faz
<code>indexar-dossie.py</code>	Indexa dossiê para RAG
<code>fatiar-obra.py</code>	Divide obra em partes
<code>pool-capitulos.py</code>	Gerencia lotes de capítulos
<code>formatar-referencias.py</code>	Formata referências ABNT
<code>renderizar-diagramas.py</code>	Renderiza Mermaid→imagem
<code>extrair-passos-praticos.py</code>	Extrai passos para playbook
<code>secoes_eita.py</code>	Parser EITA-V2
<code>revisar-e-polir-capitulos.py</code>	Revisão final
<code>empacotar-distribuicao.py</code>	Empacota para distribuição
<code>sincronizar-capas-distribuicao.py</code>	Sincroniza capas
<code>colecão.py</code>	Gerencia coleções
<code>metadados_livro.py</code>	Extrai metadados
<code>parametros_obra.py</code>	Configura parâmetros
<code>tipos_obra.py</code>	Registry de tipos

5.4 Scripts de Infraestrutura

Script	O que faz
<code>descobrir_modelos.py</code>	Detecta harness e LLMs disponíveis
<code>compilar-para-pdf.py</code>	Compila MD→PDF via Typst
<code>compilar-mega-livro.py</code>	Unifica livros
<code>qualidade-mimocode.py</code>	Verifica qualidade
<code>compensar-volume-mimocode.py</code>	Compensa volume
<code>converter-md-pdf.ps1</code>	Conversor PowerShell
<code>pdf_typst.py</code>	Wrapper Typst
<code>setup-links.ps1 / setup-links.sh</code>	Cria junctions/hardlinks
<code>sync-vscode-mcp.mjs</code>	Sincroniza MCP VS Code
<code>series_capa.py</code>	Séries de capas
<code>renomear-headers-mimocode.py</code>	Renomeia headers

6. SKILLS (26 skills)

6.1 Skills de Criação de Conteúdo

Skill	Função	Quando usar
pesquisador	Varredura web e mineração	Início de obra (Fase 1)
arquiteto	Desenha sumário macro	Após pesquisa (Fase 1)
estrategista	Decompõe capítulo em 3 pilares	Início de cada capítulo (Fase 2)
redator-eita	Escreve capítulo EITA-V2	Após estratégia (Fase 2)
redator-academico	Escreve seção ACAD (TCC/Artigo)	Para TCC e Artigos
redator-ebook	Adapta tom para e-book	Para e-books
revisor-tecnico	Audita obra inteira	Após todos capítulos (Fase 2.5)
compilador-abnt	Merge + formatação ABNT	Compilação final (Fase 3)
compilador-tcc	Compila TCC	Para TCCs
compilador-artigo	Compila artigo	Para artigos
compilador-mega-livro	Unifica livros	Para mega-livros

6.2 Skills de Marketing e Vendas

Skill	Função	Quando usar
criar-maquina-vendas	Gera projeto deployável	Após obra finalizada
sincronizar-maquina-vendas	Propaga fix do checkout em máquina existente	Máquina gerada antes do fix (checkout 404/500)
gerar-relatorio-sessao	Orquestra fechamento de sessão (relatório + testes + commit + push)	Final de toda sessão com mudanças (V5.2)

6.3 Skills de Produtividade

Skill	Função	Quando usar
caveman	Comunicação ultra-comprimida	Economia de tokens
lean-ctx	Economia de contexto	Antes de ler arquivos
headroom	Compressão de logs	Após comandos longos
rtk-memory	Memória persistente	Após resolver problemas
pre-flight-check	Validação pré-deploy	Antes de commits
calcular-gastos-sessao	Custo por sessão	Análise de custos
i-have-adhd	Foco e organização	Sessões longas

6.4 Skills Fable (Metodologia)

Skill	Função	Quando usar
fable-method	Loop de resolução de problemas	Tarefas multi-step
fable-loop	Orquestração paralela	Tarefas complexas
fable-judge	Verificação adversarial	Após completar trabalho
fable-domain	Gera skill para domínio	Novo domínio
self-learning	Captura aprendizados	Após debugging

6.5 Skills de Token Economy

Skill	Função
aplicar-token-economy	Instala infraestrutura de economia de tokens

7. SUBAGENTES (7 agentes)

Subagente	Função	Modelo
subagente-pesquisador	Varredura web, dossiê técnico	inherit
subagente-redator-capitulo	Escrita paralela de capítulos	inherit
subagente-redator-secao-tcc	Escrita de seções TCC	inherit
subagente-redator-artigo	Escrita de artigo	inherit
subagente-adaptador-ebook	Adaptação para e-book	inherit
subagente-revisor-tecnico	Auditoria técnica	inherit
subagente-ilustrador	Gera ilustrações	inherit

Modelo: Todos usam model: inherit (herdam o modelo da sessão)

8. TEMPLATES (12 templates)

8.1 Templates de Documento

Template	Formato	Para que serve
template.typ	Typst	Livro ABNT
template_tcc.typ	Typst	TCC NBR 14724
template_artigo.typ	Typst	Artigo NBR 6022
template_playbook.typ	Typst	Playbook cards
template_lead_magnet.typ	Typst	Lead Magnet
template_lead_magnet.html	HTML+CSS	Lead Magnet (Chromium)
template_deck.typ	Typst	Deck 16:9

8.2 Templates de Conteúdo

Template	Formato	Para que serve
template_eita.md	Markdown	Molde EITA-V2
capitulo_eita.md	Markdown	Capítulo individual

8.3 Templates de Infraestrutura

Template	Formato	Para que serve
payload_estado.json	JSON	Estado da esteira
reference_deck.pptx	PPTX	Referência de deck

8.4 Template de Máquina (83 arquivos)

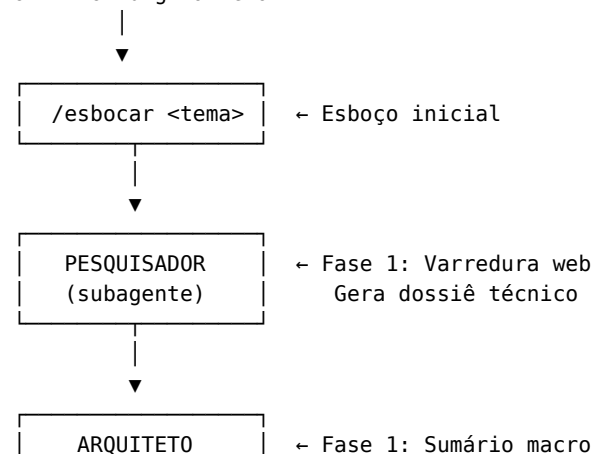
Diretório templates/maquina/ contém o projeto full-stack completo: - Frontend Next.js (29 arquivos) - Backend FastAPI (16 arquivos) - Database SQLite (2 arquivos) - Scripts (5 arquivos) - Configs (8 arquivos) - Templates de e-mail/post/DM (8 arquivos) - Deploy (3 arquivos) - Docs (6 arquivos)

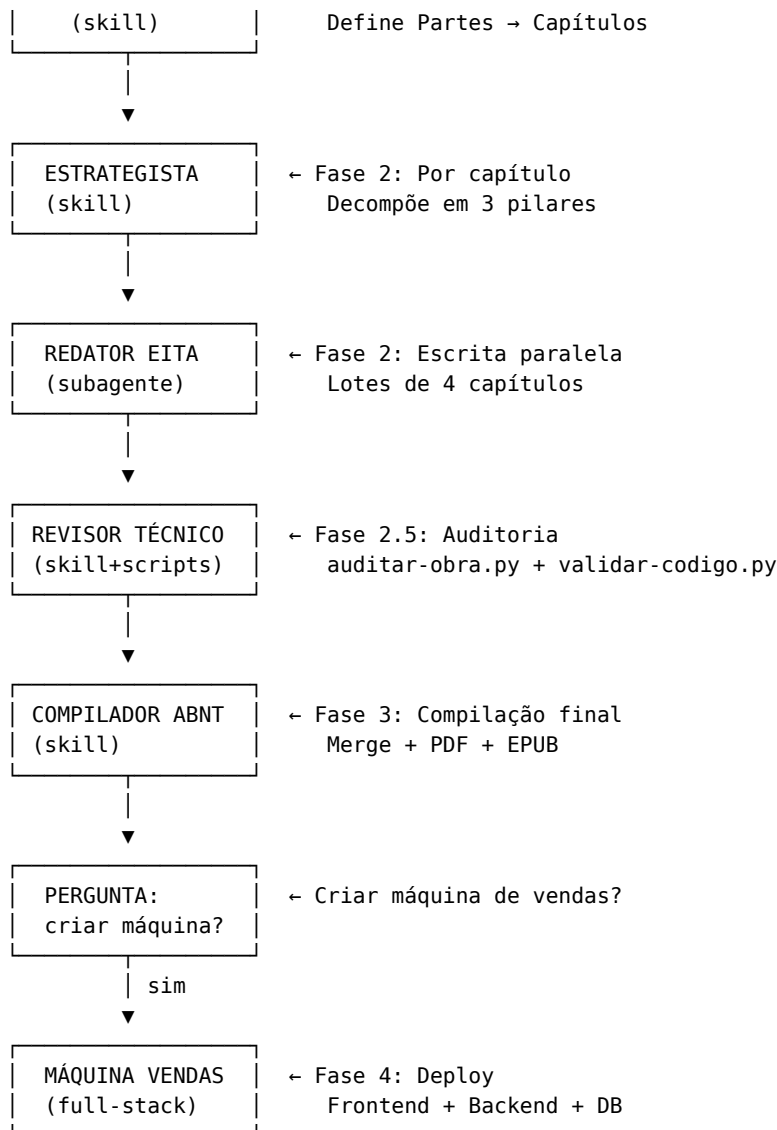
9. SPECS (9 specs)

Spec	Descreve
SPEC.md	Livro — 14 requisitos obrigatórios
SPEC_TCC.md	TCC — formatação ABNT
SPEC_ARTIGO.md	Artigo — formato IMRaD
SPEC_EBOOK.md	E-book — adaptação de tom
SPEC_PLAYBOOK.md	Playbook — passos práticos
SPEC_LEAD_MAGNET.md	Lead Magnet — material de captura
SPEC_DECK.md	Deck — apresentação
SPEC_EMAILS.md	E-mails — sequência de marketing
SPEC_MAQUINA_VENDAS.md	Máquina de Vendas — projeto deployável

10. FLUXO COMPLETO DE UMA OBRA

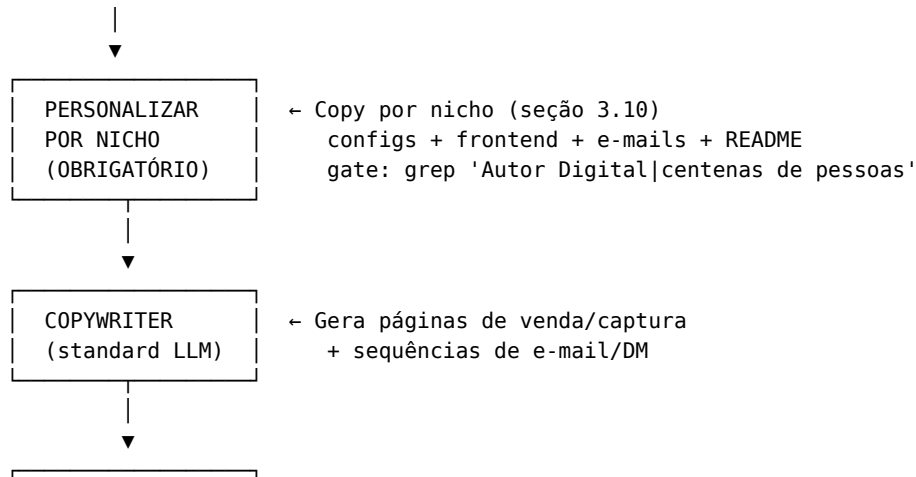
OPERADOR digita tema

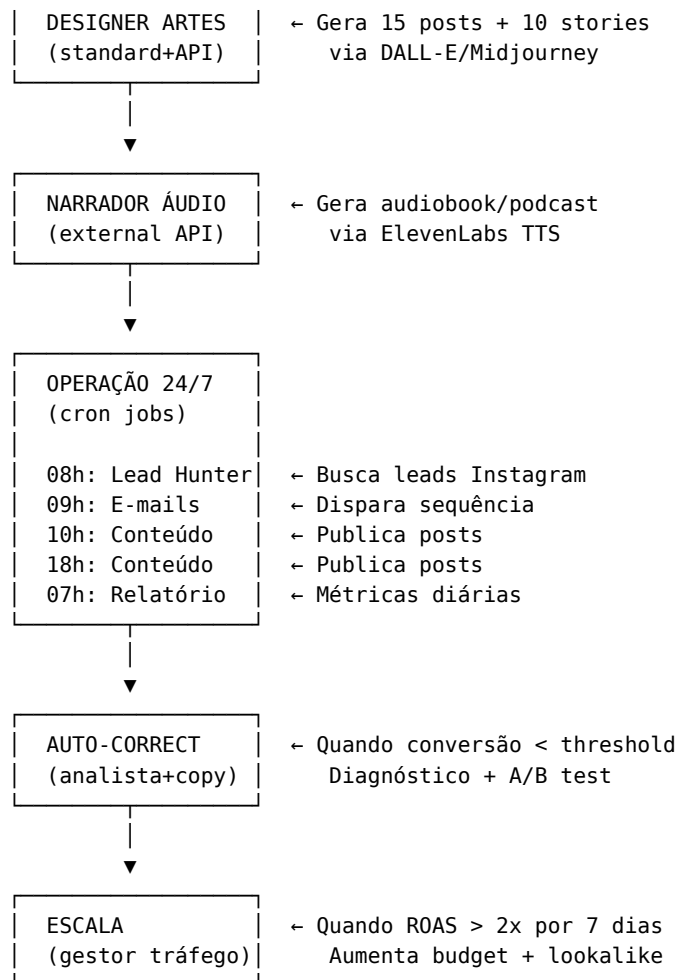




11. FLUXO DA MÁQUINA DE VENDAS

MÁQUINA CRIADA





12. CONFIGURAÇÃO E DEPLOY

12.1 Variáveis de Ambiente

```
# Banco de dados
DATABASE_URL=sqlite:///database/maquina.db

# E-mail (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=seu@email.com
SMTP_PASS=sua-senha-app

# Pagamento
STRIPE_SECRET_KEY=sk_live...
STRIPE_WEBHOOK_SECRET=whsec...

# Instagram
INSTAGRAM_ACCESS_TOKEN=EAAx...

# APIs externas (para áudio/vídeo/imagens)
ELEVENLABS_API_KEY=...
DALL_E_API_KEY=...
HEYGEN_API_KEY=...
```



```
# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_STRIPE_KEY=pk_live...

# Backend (rota /api/checkout do Next.js)
BACKEND_URL=http://127.0.0.1:8000
NEXT_PUBLIC_BACKEND_URL=http://127.0.0.1:8000
```

12.2 Deploy Docker

```
cd marketing/maquinas/{slug}
docker-compose up -d
```

Sobe 6 serviços: - frontend (Next.js :3000) - backend (FastAPI :8000) - worker-emails (processador de e-mails) - worker-leads (lead hunter) - monitor (funnel monitor) - nginx (proxy reverso :80)

12.3 Deploy Vercel + Railway

```
# Frontend → Vercel
cd frontend
vercel deploy
```

```
# Backend → Railway
cd backend
railway up
```

12.4 Deploy VPS

```
bash scripts/deploy.sh
```

13. ROTEAMENTO DE LLMs

13.1 Harnesses Suportados

Harness	Lite	Standard	Pro
MiMoCode	mimo-v2.5-lite	mimo-v2.5	mimo-v2.5-pro
Claude Code	claude-haiku	claude-sonnet	claude-opus
Gemini CLI	gemini-flash	gemini-2.5-pro	gemini-ultra
Grok	grok-2-mini	grok-2	grok-3-heavy
Kiro	titan-lite	claude-sonnet-v2	claude-opus-v2
OMP	mistral-small	mistral-large	deepseek-r1

13.2 Tarefas por Tier

Tier	Tarefas	Custo
lite	Scoring, classificação, templates, dedup	Muito baixo
standard	Copy, e-mails, páginas, análise, prompts de arte	Médio
pro	Estratégia, diagnóstico, otimização complexa	Alto
external_api	Áudio (TTS), imagem (DALL-E), vídeo (HeyGen)	Variável

13.3 Detecção Automática

`python scripts/descobrir_modelos.py`

Detecta harness, lista modelos disponíveis, roteia cada tarefa para o modelo mais barato.

14. TROUBLESHOOTING

Problema	Causa	Solução
“Obra não encontrada”	Slug incorreto	Verificar <code>ls output/livros/</code>
“Capítulo truncado”	Token limit atingido	<code>auditar-obra.py</code> detecta e corrige
“Código com erro”	Sintaxe inválida	<code>validar-codigo.py</code> detecta
“PDF não gera”	Typst não instalado	<code>pip install typst</code> ou usar Pandoc
“EPUB falha”	Pandoc ausente	<code>choco install pandoc</code>
“Build Next.js falha”	Node/npm ausente	<code>npm install</code> no frontend
“FastAPI não inicia”	Dependências faltando	<code>pip install -r requirements.txt</code>
“SQLite corrompido”	Arquivo deletado	Rodar migrations novamente
“E-mails não enviam”	SMTP não configurado	Configurar <code>config/email.json</code>
“Leads não aparecem”	Instagram token ausente	Configurar <code>.env</code>
“Checkout dá 404”	Rota <code>/api/checkout</code> removida	Regerar do template (rota nasce na geração)
“Checkout sempre dá erro”	Form html posta urlencoded; rota exige JSON	<code>checkout/page.tsx</code> usa fetch JSON com nome/e-mail (padrão do template)
“Site com copy genérica”	Máquina não personalizada	Seguir seção 3.10 (gate R12)
“Acentos/emojis quebrados no console”	Terminal Windows cp1252	Scripts da fábrica usam <code>console_utf8()/sys.stdout.reconfigure</code> (regra 11 do AGENTS.md)
“Gasto tokens alto”	Modelo errado para tarefa	<code>python scripts/descobrir_modelos.py</code>
“Build Typst falha”	Template com erro	Verificar <code>.typ</code> no editor
“Diagrama não renderiza”	Mermaid inválido	<code>renderizar-diagramas.py --validar</code>

15. ESTRUTURA DE OUTPUT

```
output/
├─ livros/           Livros gerados
│   └─ {slug}/
│       ├─ capitulos/  Capítulos em Markdown
│       └─ pesquisa/   Dossiê técnico
```

